

Algorithms and Data Structures

Example Author · Fall 2026

Contents

1. Getting started	2
1.1. Headings	3
1.1.1. A third-level heading	3
1.2. Running header	3
2. Components	3
2.1. Notes	4
2.2. Callouts	4
2.3. Tables	5
2.4. Code	5
2.5. Math	6
3. Sorting algorithms	6
3.1. Binary search	7
3.2. Insertion sort	7
3.3. Heapsort	8
3.4. Comparison	8
4. Theming	8
5. Accessibility	9

CHAPTER 1

Getting started

This document is a tour of the **lambda-notes** template. Every page you see was produced by the template’s styling: the chapter titles, the running header at the top of the page, the tables, the code blocks, and the algorithm figures.

Links come in two colors. External links like [the Typst website](#) use the theme color, while internal links like [the components chapter](#) or [the sorting chapter](#) are shown in green, so you can tell navigation apart from the outside world at a glance. Plain references such as Section 3 work too.

1.1. Headings

Level-1 headings open a new chapter on a fresh page, with a small tracked “Chapter N” label, a large title, and a rule. Level-2 and level-3 headings are numbered and tinted with the theme color.

1.1.1. A third-level heading

Deeper headings keep the same numbering scheme, e.g. `1.1.1`.

1.2. Running header

Starting from the second page of a chapter, the header shows the current chapter on the left and the current section on the right. Look at the top of the next pages to see it change as sections go by.

CHAPTER 2

Components

2.1. Notes

A `note` is a plain, undecorated aside. It ignores the theme color, so it stays neutral no matter how the document is styled.

Notes can hold anything: **bold**, *italic*, `inline code`, and math like $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.

2.2. Callouts

Callouts take an optional `title` and a `color`, which makes them handy for building your own vocabulary of boxes.

Definition

A **graph** is a pair $G = (V, E)$ where V is a set of vertices and $E \subseteq V \times V$ is a set of edges.

Theorem

Every comparison-based sorting algorithm performs $\Omega(n \log n)$ comparisons in the worst case.

Tip

Use a hash map when you need $O(1)$ average-case lookups by key.

Warning

Quicksort degrades to $O(n^2)$ on already-sorted input if the pivot is always the first element.

Common mistake

Forgetting the base case of a recursive function leads to infinite recursion.

The title is optional: without it, a callout is just a tinted box.

2.3. Tables

The header row picks up a tint of the theme color, body rows are zebra-striped, and the first column is left-aligned while the others are centered.

Data structure	Access	Search	Insert
Array	$O(1)$	$O(n)$	$O(n)$
Linked list	$O(n)$	$O(n)$	$O(1)$
Hash table	—	$O(1)$	$O(1)$
Binary search tree	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	$O(1)$	$O(n)$	$O(\log n)$

Table 1: Average-case complexity of common data structures.

Table 1 summarizes the trade-offs; figures are numbered and can be referenced like any other label.

2.4. Code

Inline code such as `let x = 42` gets a small highlighted box. Code blocks are rendered with [codly](#), with a language icon for each snippet.

```
def fibonacci(n: int) -> int:
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

```
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

```
fn is_prime(n: u64) -> bool {
    if n < 2 {
```

```
        return false;
    }
    (2..).take_while(|i| i * i <= n).all(|i| n % i != 0)
}
```

```
public class Stack<T> {
    private final List<T> items = new ArrayList<>();

    public void push(T item) { items.add(item); }
    public T pop() { return items.remove(items.size() - 1); }
}
```

2.5. Math

Math is plain Typst math, set in the body text style:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$$

CHAPTER 3

Sorting algorithms

Pseudocode is written with the [algorithmic](#) package, styled to match the rest of the document.

3.1. Binary search

Algorithm 1: Binary search

```
1  procedure BINARY-SEARCH( $A$ ,  $target$ )
2     $lo \leftarrow 0$ 
3     $hi \leftarrow |A| - 1$ 
4    while  $lo \leq hi$  do
5       $mid \leftarrow \lfloor \frac{lo + hi}{2} \rfloor$ 
6      if  $A[mid] = target$  then
7        return  $mid$ 
8      end
9      else if  $A[mid] < target$  then
10        $lo \leftarrow mid + 1$ 
11      end
12      else
13        $hi \leftarrow mid - 1$ 
14      end
15    end
16    return  $-1$ 
17 end
```

Algorithm 1 runs in $O(\log n)$ time on a sorted array of size n .

3.2. Insertion sort

Algorithm 2: Insertion sort

```
1  procedure INSERTION-SORT( $A$ ,  $n$ )
2    for  $i = 2$  to  $n$  do
3       $key \leftarrow A[i]$ 
4       $\triangleright$  Insert  $A[i]$  into the sorted prefix  $A[1..i - 1]$ 
5       $j \leftarrow i - 1$ 
6      while  $j > 0$  and  $A[j] > key$  do
7         $A[j + 1] \leftarrow A[j]$ 
8         $j \leftarrow j - 1$ 
9      end
10      $A[j + 1] \leftarrow key$ 
```

```

11 | end
12 end

```

Complexity

Algorithm 2 is $O(n^2)$ in the worst case but $O(n)$ on nearly-sorted input, which makes it a good fallback for small subarrays.

3.3. Heapsort

Algorithm 3: Heapsort

```

1  procedure HEAPSORT( $A, n$ )
2    BUILD-MAX-HEAP( $A, n$ )
3    for  $i = n$  downto 2 do
4      ▷ Move the current maximum to the end
5       $A[1], A[i] \leftarrow A[i], A[1]$ 
6       $A.\text{heap-size} \leftarrow A.\text{heap-size} - 1$ 
7      MAX-HEAPIFY( $A, 1$ )
8    end
9  end

```

3.4. Comparison

Algorithm	Best	Average	Worst
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Back to [the components chapter](#), or read about theming in the next chapter.

CHAPTER 4

Theming

The whole look is driven by a single `color` argument: chapter titles, rules, headings, links, and table headers all derive from it. Try swapping `blue` at the top of this file for `rgb("#8E24AA")`, `olive`, or `none` for a plain black-and-white document.

```
#show: lambda-notes.with(  
  title: "Networking",  
  color: rgb("#1E88E5"),  
  chapter-label: [Capitolo],  
  outline-title: [Indice],  
)
```

`chapter-label` and `outline-title` let you localize the template, for example into Italian as shown above.

CHAPTER 5

Accessibility

All of the accessibility settings for the document can be configured through the `#show: lambda-notes.with` block.

```
#show: lambda-notes.with(  
  title: "Algorithms and Data Structures",  
  author: "Example Author",  
  subject: "lambda-notes showcase",  
  keywords: ("algorithms", "data structures", "typst", "template"),  
)
```

These attributes will show up in the PDF metadata.